Architechture de Projet:

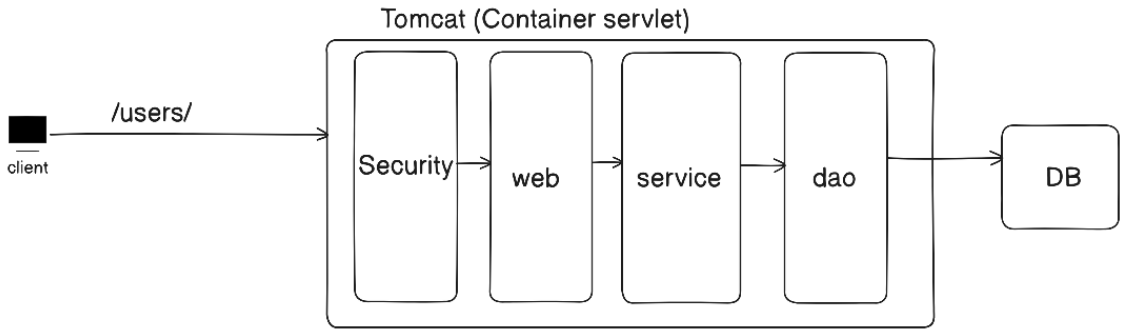
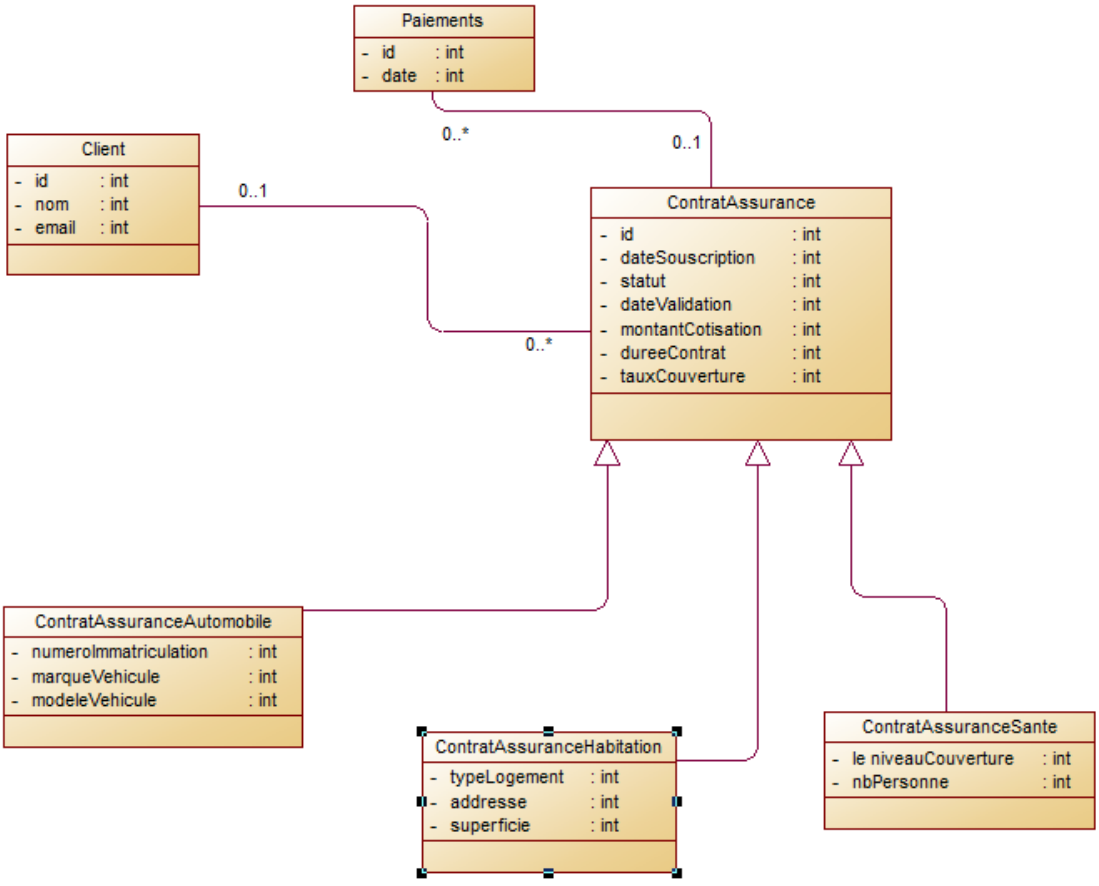


Diagramme De classe :



2- couche Dao

a- Creer les entitees

Client:

```java
package ma.enset.examjee.entity;


import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

import java.util.List;

@Entity
@Data @AllArgsConstructor @NoArgsConstructor
public class Client {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String nom;
    private String email;
    @OneToMany(mappedBy = "client")
    private List<ContratAssurance> contratAssuranceList;
}
```

ContartAssurance:

```java
package ma.enset.examjee.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import ma.enset.examjee.entity.enumiration.StatusContrat;

import java.time.LocalDate;
import java.util.List;

@Entity
@Data @AllArgsConstructor @NoArgsConstructor
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "type", length = 30)
public class ContratAssurance {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDate dateSouscription;
    @Enumerated(value = EnumType.STRING)
    private StatusContrat statusContrat;
    private LocalDate dateValidation;
    private double montantCotisation;
    private int dureeContart;
    private double tauxCouverture;
    @ManyToOne
    private Client client;
    @OneToMany(mappedBy = "contartAssurance")
    private List<Paiements> paiementsList;
}
```

ContartAssuranceSante:

```
package ma.enset.examjee.entity;

import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import lombok.Data;
import ma.enset.examjee.entity.enumiration.NiveauCouverture;
import ma.enset.examjee.entity.enumiration.TypeLogement;

@Entity
@Data
@DiscriminatorValue("SANTE")
public class ContartAssuranceSante extends ContratAssurance{
   @Enumerated(value = EnumType.STRING)
   private NiveauCouverture niveauCouverture;
   private int nbPersonne;
}
```

ContartAssuranceHabitation :

```
package ma.enset.examjee.entity;

import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;
import jakarta.persistence.EnumType;
import jakarta.persistence.Enumerated;
import lombok.Data;
import ma.enset.examjee.entity.enumiration.TypeLogement;

@Entity
@Data
@DiscriminatorValue("HABITATION")
public class ContartAssuranceHabitation extends ContratAssurance{
   @Enumerated(value = EnumType.STRING)
   private TypeLogement typeLogement;
   private String addresse;
   private double superficie;
}
```

ContratAssuranceAutomobile:

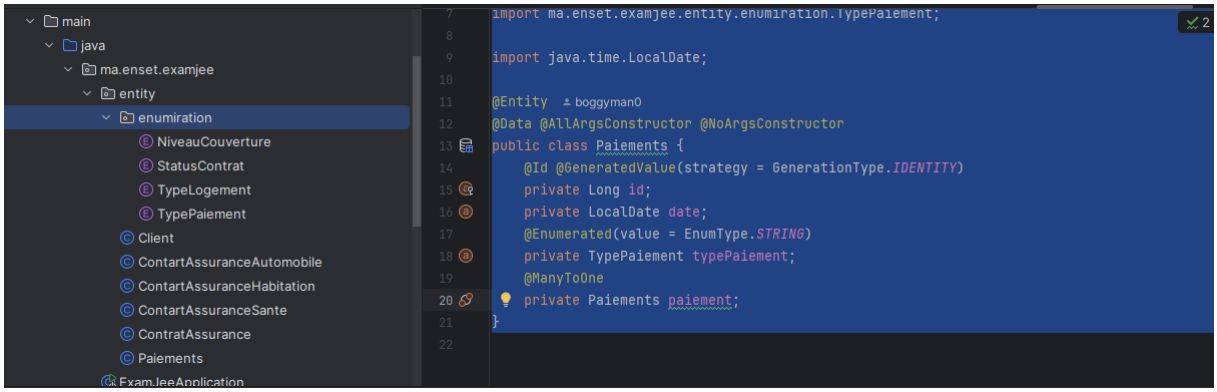
```
package ma.enset.examjee.entity;

import jakarta.persistence.DiscriminatorValue;
import jakarta.persistence.Entity;
import lombok.Data;

@Entity
@Data
@DiscriminatorValue("AUTOMOBILE")
public class ContartAssuranceAutomobile extends ContratAssurance{

    private long numeroImmatriculation;
    private String marque;
    private String modele;
```

```
}
```

Paiements:

```
package ma.enset.examjee.entity;

import jakarta.persistence.*;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import ma.enset.examjee.entity.enumiration.TypePaiement;

import java.time.LocalDate;

@Entity
@Data @AllArgsConstructor @NoArgsConstructor
public class Paiements {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private LocalDate date;
    @Enumerated(value = EnumType.STRING)
    private TypePaiement typePaiement;
    @ManyToOne
    private ContratAssurance contartAssurance;
}
```

Structure de Code :



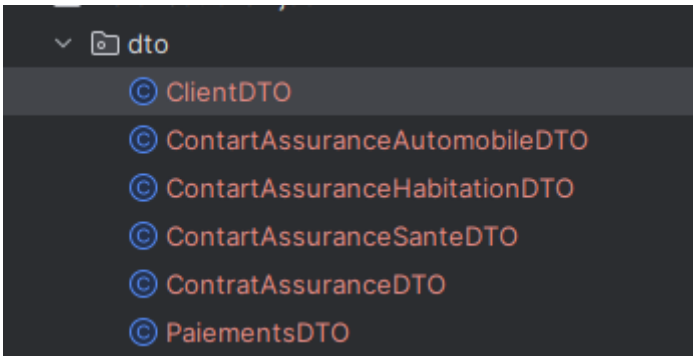
b- Les interfaces Jpa Repository

```
c- package ma.enset.examjee.repository;

   import ma.enset.examjee.entity.Client;
   import org.springframework.data.jpa.repository.JpaRepository;

   public interface ClientRepo extends JpaRepository<Client, Long> {
   }
```

```java
package ma.enset.examjee.repository;

import ma.enset.examjee.entity.Paiements;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ContartAssuranceRepo extends JpaRepository<Paiements,
Long> {
}
```

```java
package ma.enset.examjee.repository;

import ma.enset.examjee.entity.Paiements;
import org.springframework.data.jpa.repository.JpaRepository;

public interface PaiementRepo extends JpaRepository<Paiements, Long> {
}
```

J’ai ajouter la couche DTO :



Et j’ai créer les mapper :

```java
package ma.enset.examjee.mapper;

import ma.enset.examjee.dto.ClientDTO;
import ma.enset.examjee.entity.Client;
import org.springframework.beans.BeanUtils;
import org.springframework.context.annotation.Bean;
import org.springframework.stereotype.Component;

@Component
public class ClientMapper {

    public Client formClientDTO(ClientDTO clientDTO) {
        Client client = new Client();
        BeanUtils.copyProperties(clientDTO, client);
        return client;
    }

    public ClientDTO fromClient(Client client) {
        ClientDTO clientDTO = new ClientDTO();
        BeanUtils.copyProperties(client, clientDTO);
        return clientDTO;
    }
}
```

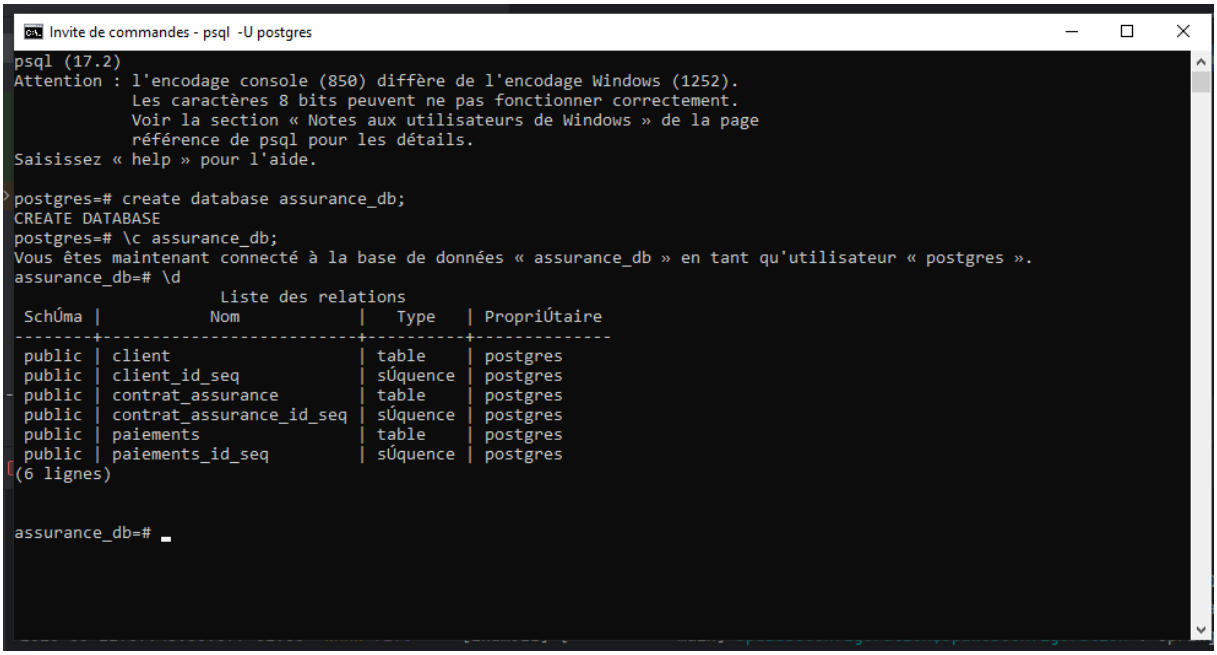
```java
package ma.enset.examjee.mapper;

import ma.enset.examjee.dto.PaiementsDTO;
import ma.enset.examjee.entity.Paiements;
import org.springframework.beans.BeanUtils;
import org.springframework.stereotype.Component;

@Component
public class PaiementMapper {

    public Paiements fromPaiementDTO(PaiementsDTO paiementsDTO) {
        Paiements paiements = new Paiements();
        BeanUtils.copyProperties(paiementsDTO, paiements);
        return paiements;
    }

    public PaiementsDTO fromPaiement(Paiements paiements) {
        PaiementsDTO paiementsDTO = new PaiementsDTO();
        BeanUtils.copyProperties(paiements, paiementsDTO);
        return paiementsDTO;
    }


}
```

```java
package ma.enset.examjee.mapper;

import ma.enset.examjee.dto.ContratAssuranceAutomobileDTO;
import ma.enset.examjee.dto.ContratAssuranceHabitationDTO;
import ma.enset.examjee.dto.ContratAssuranceSanteDTO;
import ma.enset.examjee.entity.ContratAssuranceAutomobile;
import ma.enset.examjee.entity.ContratAssuranceHabitation;
import ma.enset.examjee.entity.ContratAssuranceSante;
import org.springframework.beans.BeanUtils;
import org.springframework.stereotype.Component;
```

```java
@Component
public class ContratAssuaranceMapper {

    public ContratAssuranceSante
fromContratSanteDTO(ContratAssuranceSanteDTO contratAssuranceSanteDTO) {
        ContratAssuranceSante contratAssuranceSante = new
ContratAssuranceSante();
        BeanUtils.copyProperties(contratAssuranceSanteDTO,
contratAssuranceSante);
        return contratAssuranceSante;
    }

    public ContratAssuranceSanteDTO fromContratSante(ContratAssuranceSante
contratAssuranceSante) {
        ContratAssuranceSanteDTO contratAssuranceSanteDTO = new
ContratAssuranceSanteDTO();
        BeanUtils.copyProperties(contratAssuranceSante,
contratAssuranceSanteDTO);
        return contratAssuranceSanteDTO;
    }

    public ContratAssuranceAutomobile
fromContratAutomobileDTO(ContratAssuranceAutomobileDTO
contratAssuranceAutomobileDTO) {
        ContratAssuranceAutomobile contratAssuranceAutomobile = new
ContratAssuranceAutomobile();
        BeanUtils.copyProperties(contratAssuranceAutomobileDTO,
contratAssuranceAutomobile);
        return contratAssuranceAutomobile;
    }

    public ContratAssuranceAutomobileDTO
fromContratAutomobile(ContratAssuranceAutomobile
contratAssuranceAutomobile) {
        ContratAssuranceAutomobileDTO contratAssuranceAutomobileDTO = new
ContratAssuranceAutomobileDTO();
        BeanUtils.copyProperties(contratAssuranceAutomobile,
contratAssuranceAutomobileDTO);
        return contratAssuranceAutomobileDTO;
    }


    public ContratAssuranceHabitation
fromContratHabitationDTO(ContratAssuranceHabitationDTO
contratAssuranceHabitationDTO) {
        ContratAssuranceHabitation contratAssuranceHabitation = new
ContratAssuranceHabitation();
        BeanUtils.copyProperties(contratAssuranceHabitationDTO,
contratAssuranceHabitation);
        return contratAssuranceHabitation;
    }

    public ContratAssuranceHabitationDTO
fromContratHabitation(ContratAssuranceHabitation
contratAssuranceHabitation) {
        ContratAssuranceHabitationDTO contratAssuranceHabitationDTO = new
ContratAssuranceHabitationDTO();
        BeanUtils.copyProperties(contratAssuranceHabitation,
contratAssuranceHabitationDTO);
        return contratAssuranceHabitationDTO;
```

```
    }

}
```

Apres avoir executer le projet sur le port 9095 les tables sont bien créer dans la base de donees



d- tester l'application :

```
e- package ma.enset.examjee;

   import lombok.RequiredArgsConstructor;
   import ma.enset.examjee.dto.ClientDTO;
   import ma.enset.examjee.dto.ContratAssuranceHabitationDTO;
   import ma.enset.examjee.entity.Client;
   import ma.enset.examjee.entity.ContratAssuranceHabitation;
   import ma.enset.examjee.entity.enumiration.StatusContrat;
   import ma.enset.examjee.entity.enumiration.TypeLogement;
   import ma.enset.examjee.mapper.ClientMapper;
   import ma.enset.examjee.mapper.ContratAssuaranceMapper;
   import ma.enset.examjee.mapper.PaiementMapper;
   import ma.enset.examjee.repository.ClientRepo;
   import ma.enset.examjee.repository.ContratAssuranceRepo;
   import org.springframework.boot.CommandLineRunner;
   import org.springframework.boot.SpringApplication;
   import org.springframework.boot.autoconfigure.SpringBootApplication;
   import org.springframework.context.annotation.Bean;

   import java.time.LocalDate;
   import java.util.List;
   import java.util.stream.Stream;

   @SpringBootApplication
   @RequiredArgsConstructor
   public class ExamJeeApplication {

       private final ClientMapper clientMapper;
       private final ContratAssuaranceMapper contratAssuaranceMapper;
```

```
    public static void main(String[] args) {
        SpringApplication.run(ExamJeeApplication.class, args);
    }


    @Bean
    CommandLineRunner runner(ClientRepo clientRepo, ContratAssuranceRepo
contratAssuranceRepo, PaiementMapper paiementMapper) {
        return args -> {

            Stream.of("omar", "hassan", "yassin").forEach(client -> {
                ClientDTO clientDTO = new ClientDTO();
                clientDTO.setNom(client);
                clientDTO.setEmail(client+"@gmail.com");
                clientRepo.save(clientMapper.formClientDTO(clientDTO));
            });

            List<Client> list = clientRepo.findAll();
            list.stream().forEach(client -> {
                ContratAssuranceHabitationDTO
contratAssuranceHabitationDTO = new ContratAssuranceHabitationDTO();

contratAssuranceHabitationDTO.setAddresse("address"+Math.random()*10+"ma
rrakech "+ client.getNom());

contratAssuranceHabitationDTO.setSuperficie(Math.random() * 100);

contratAssuranceHabitationDTO.setTypeLogement(TypeLogement.APPARTEMENT);

contratAssuranceHabitationDTO.setStatusContrat(StatusContrat.EN_COURS);

contratAssuranceHabitationDTO.setDateValidation(LocalDate.now().plusYear
s(2));
                contratAssuranceHabitationDTO.setDureeContart(300);

contratAssuranceHabitationDTO.setTauxCouverture(Math.random());

contratAssuranceHabitationDTO.setDateSouscription(LocalDate.now());

contratAssuranceHabitationDTO.setMontantCotisation(Math.random() *
1000);
                ContratAssuranceHabitation contratAssuranceHabitation =
contratAssuaranceMapper.fromContratHabitationDTO(contratAssuranceHabitat
ionDTO);
                contratAssuranceHabitation.setClient(client);
                contratAssuranceRepo.save(contratAssuranceHabitation);
            });


        };
    }
}
```

3- ajouter la couche service :

- Client Service :

```
- package ma.enset.examjee.service.serviceImpl;

  import lombok.RequiredArgsConstructor;
  import ma.enset.examjee.dto.ClientDTO;
  import ma.enset.examjee.dto.ContratAssuranceDTO;
  import ma.enset.examjee.entity.*;
  import ma.enset.examjee.exception.ClientNotFoundException;
  import ma.enset.examjee.mapper.ClientMapper;
  import ma.enset.examjee.mapper.ContratAssuaranceMapper;
  import ma.enset.examjee.repository.ClientRepo;
  import ma.enset.examjee.repository.ContratAssuranceRepo;
  import ma.enset.examjee.service.IClientService;
  import org.springframework.stereotype.Service;

  import java.util.List;

  @Service
  @RequiredArgsConstructor
  public class ClientService implements IClientService {

      private final ClientRepo clientRepo;
      private final ClientMapper clientMapper;
      private final ContratAssuaranceMapper contratAssuaranceMapper;

      @Override
      public ClientDTO createClient(ClientDTO clientDTO) {
          Client client = clientMapper.formClientDTO(clientDTO);
          return clientMapper.fromClient(clientRepo.save(client));
      }

      @Override
      public List<ClientDTO> getAllClients() {
          List<Client> clients = clientRepo.findAll();
          return
  clients.stream().map(clientMapper::fromClient).toList();
      }

      @Override
      public ClientDTO getClientByNom(String nom) {
          Client client = clientRepo.findByNom(nom);
          return clientMapper.fromClient(client);
      }

      @Override
      public List<ContratAssuranceDTO> getListContratbyClientId(long
  clientId) {
          Client client = clientRepo.findById(clientId).orElseThrow(()
  -> new ClientNotFoundException("client with id " + clientId + " not
  found"));

          List<ContratAssurance> list =
  client.getContratAssuranceList();

          return list.stream().map(contrat -> {
              if (contrat instanceof ContratAssuranceAutomobile
  contratAssuranceAutomobile) {
                  return
  contratAssuaranceMapper.fromContratAutomobile(contratAssuranceAutomob
  ile);
```

```
                }else if (contrat instanceof ContratAssuranceHabitation
    contratAssuranceHabitation) {
                    return
    contratAssuaranceMapper.fromContratHabitation(contratAssuranceHabitat
    ion);
                }else {
                    ContratAssuranceSante contratAssuranceSante =
    (ContratAssuranceSante) contrat;
                    return
    contratAssuaranceMapper.fromContratSante(contratAssuranceSante);
                }
            }).toList();
        }
    }
    @Override
public void deleteClient(long clientId) {
    Client client = clientRepo.findById(clientId).orElseThrow(() -> new
ClientNotFoundException("not found"));
    client.setActive(false);
    clientRepo.save(client);
}
```

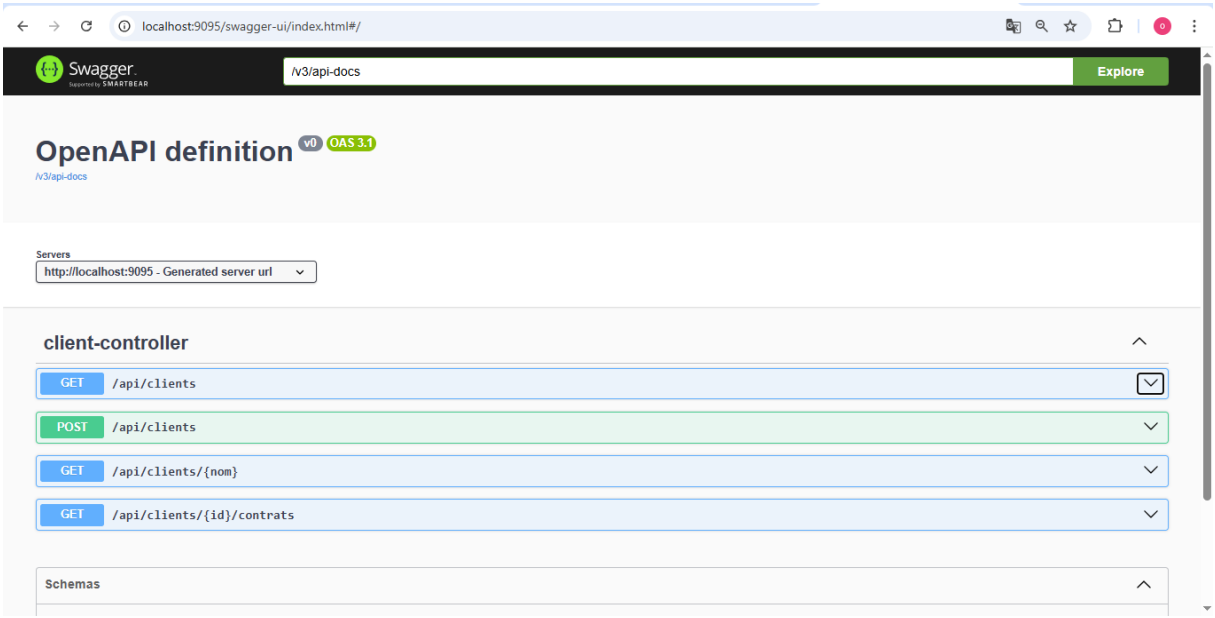
4- realisation de la couche controller

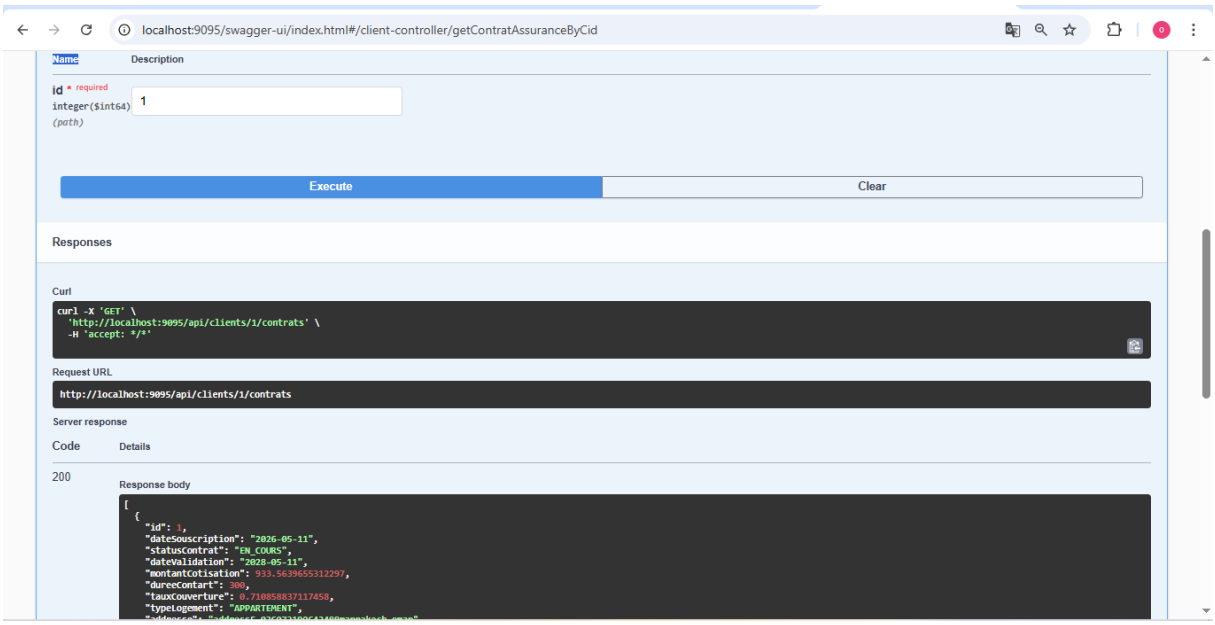
- ClientController :

```
-   package ma.enset.examjee.controller;

    import lombok.RequiredArgsConstructor;
    import ma.enset.examjee.dto.ClientDTO;
    import ma.enset.examjee.dto.ContratAssuranceDTO;
    import ma.enset.examjee.service.IClientService;
    import org.springframework.http.HttpStatus;
    import org.springframework.http.ResponseEntity;
    import org.springframework.web.bind.annotation.*;

    import java.util.List;

    @RestController
    @RequestMapping("/api/clients")
    @RequiredArgsConstructor
    public class ClientController {
        private final IClientService clientService;


        @PostMapping
        public ResponseEntity<ClientDTO> createClientDto(@RequestBody
    ClientDTO clientDTO) {
            return new
    ResponseEntity<>(clientService.createClient(clientDTO),
    HttpStatus.CREATED);
        }

        @GetMapping
        public ResponseEntity<List<ClientDTO>> getAllClients() {
            return new ResponseEntity<>(clientService.getAllClients(),
    HttpStatus.OK);
        }

        @GetMapping("{nom}")
        public ResponseEntity<ClientDTO> getClientByNom(@PathVariable
```

```
    String nom) {
            return new
    ResponseEntity<>(clientService.getClientByNom(nom), HttpStatus.OK);
        }

        @GetMapping("/{id}/contrats")
        public ResponseEntity<List<ContratAssuranceDTO>>
    getContratAssuranceByCid(@PathVariable long id) {
            return new
    ResponseEntity<>(clientService.getListContratbyClientId(id),
    HttpStatus.OK);
        }

    }

     @DeleteMapping("{id}")
public void deleteClient(@PathVariable long id) {
    clientService.deleteClient(id);
}
```

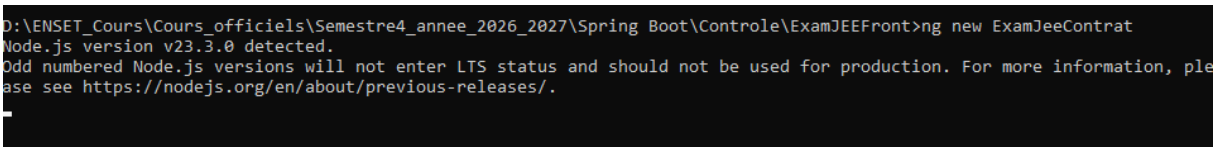
- j'ai tester mes endpoints utilisant swagger sa marche :





5- proposant une application front end :

- créer le projet front end utilisant
  - ng new ExamJeeContrat



- ajouter client
- supprimer client
- visualiser les contarts de chaque client